

# SPRAWOZDANIE Z PROJEKTU

System Zarządzania Wydarzeniami Akademickimi

Zuzanna Szymerkowska

# 1. Cel i założenia projektu

Przedmiotem projektu było zaprojektowanie, zaimplementowanie i uruchomienie kompletnej, wielowarstwowej aplikacji webowej służącej do zarządzania wydarzeniami na Akademii Marynarki Wojennej. Głównym założeniem systemu było umożliwienie użytkownikom bieżącego przeglądania harmonogramu, dodawania nowych wydarzeń poprzez intuicyjny formularz oraz usuwania nieaktualnych wpisów w czasie rzeczywistym.

Aplikacja została zaprojektowana w architekturze rozproszonej, zapewniającej całkowitą separację warstwy prezentacji (Frontend) od warstwy logiki biznesowej i dostępu do danych (Backend). Dodatkowym celem edukacyjnym było przygotowanie serwera do elastycznej obsługi dwóch niezależnych paradygmatów bazodanowych: relacyjnego (MySQL) oraz obiektowego (NoSQL / MongoDB).

## 2. Szczegółowy opis aplikacji

Aplikacja udostępnia użytkownikowi końcowemu dynamiczny interfejs typu SPA. Dzięki temu nawigacja pomiędzy poszczególnymi modułami oraz operacje na danych odbywają się bez konieczności przeładowywania całej strony w przeglądarce.

Główne moduły funkcjonalne systemu:

- Panel główny (Lista Wydarzeń): Centralny punkt aplikacji prezentujący tabelaryczne zestawienie wszystkich zaplanowanych wydarzeń. Dane są dynamicznie pobierane z bazy danych przy inicjalizacji komponentu. Każdy rekord zawiera tytuł, sformatowaną datę, szczegółowy opis oraz dedykowany przycisk akcji.
- Moduł rejestracji (Formularz): Bezpieczny komponent wejściowy zbierający dane od użytkownika. Zawiera walidację pól formularza, automatyczne mapowanie danych na model strukturalny w TypeScript oraz asynchroniczne wystanie obiektu do serwera za pomocą protokołu HTTP.
- System usuwania danych: Bezpośrednia integracja tabeli z bazą, pozwalająca na natychmiastowe usunięcie wybranego rekordu za pomocą jego unikalnego klucza (ID). Operacja zabezpieczona jest systemowym oknem potwierdzenia (Confirm dialog), co minimalizuje ryzyko przypadkowego skasowania danych.

### 3. Opis zastosowanych technologii

Projekt został zrealizowany w oparciu o nowoczesny stos technologiczny, gwarantujący wysoką wydajność, bezpieczeństwo oraz pełną skalowalność kodu:

- Angular (v17+) / TypeScript (Frontend): Szkielet aplikacji oparty o komponenty autonomiczne (Standalone Components). Zastosowano asynchroniczne strumienie danych (RxJS) oraz moduł HttpClient do komunikacji z zewnętrznym API. Logika biznesowa została całkowicie odseparowana od widoku za pomocą dedykowanej warstwy usług (Services). Za reaktywność interfejsu i błyskawiczne renderowanie zmian odpowiada biblioteka Zone.js.
- Node.js / Express.js (Backend): Środowisko uruchomieniowe oraz framework odpowiedzialny za wystawienie RESTful API. Serwer implementuje architekturę sterowaną zdarzeniami, obsługuje parsowanie obiektów JSON, konfigurację nagłówków CORS (umożliwiających bezpieczną komunikację między różnymi portami localhost) oraz rutowanie żądań HTTP (GET, POST, DELETE).
- MySQL (Baza Relacyjna): Główny magazyn danych projektu. Przechowuje informacje w strukturyzowanej tabeli wydarzenia z automatycznie inkrementowanym kluczem głównym (ID). Połączenie z serwerem Node.js realizowane jest za pomocą wydajnego, natywnego sterownika mysql2, wykonującego asynchroniczne zapytania strukturalne.
- MongoDB (Baza NoSQL - Alternatywna): W celach demonstracyjnych i architektonicznych backend został wyposażony w drugi, niezależny serwer deweloperski współpracujący z MongoDB (poprzez mapowanie obiektowo-dokumentowe Mongoose). Pozwala to na natychmiastowe przełączenie aplikacji z zapisu relacyjnego na obiektowy (dokumentowy) poprzez prostą zmianę portu we frontendzie.
- Bootstrap (v5): Framework CSS wykorzystany do ostylowania warstwy wizualnej. Zapewnia pełną responsywność interfejsu (RWD), estetyczny wygląd tabel, formularzy, kart typu Card oraz odpowiednie cieniowania i typografię zgodną z nowoczesnymi standardami UI/UX.

## 4. Specyfikacja portów i środowiska sieciowego

Podczas działania w środowisku deweloperskim poszczególne komponenty architektury komunikują się ze sobą w pętli zwrotnej za pomocą protokołu TCP/IP na przypisanych portach lokalnych:

`http://localhost:4200` — Warstwa klienta (Angular SPA)

`http://localhost:3000` — Serwer główny API (Node.js + MySQL)

`http://localhost:3001` — Serwer alternatywny API (Node.js + MongoDB / MEAN)

`http://localhost/phpmyadmin` — Graficzny panel zarządzania bazą danych MySQL

## 5. Podsumowanie i wnioski

Wszystkie założenia projektowe zostały w pełni zrealizowane. Aplikacja pomyślnie przeszła testy integracyjne, wykazując poprawność przesyłu informacji w pełnym cyklu CRUD. Utworzenie modularnej architektury pozwoliło na elastyczne zarządzanie kodem, co udowodniła łatwość adaptacji systemu zarówno do bazy MySQL, jak i MongoDB.